ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 1ST YEAR

PROGRAMMING FUNDAMENTALS

PAPER 1 · PRACTICE A

MARKS: 100 **TIME: 3 hours**

BSc Computer Science, 1st year, semester 1 final examination. Section A: objective questions (10) · Section B: concepts, code tracing, debugging, writing Python functions and programs, searching and sorting (90).

Written by Academic Central for practice, following first-year Computer Science syllabuses at South African universities (introduction to programming and algorithms). It is not a university examination paper.

INSTRUCTIONS AND INFORMATION

1. This question paper consists of SIX questions. Answer ALL the questions.
2. Write all code in Python 3. Use clear variable names and the correct indentation.
3. Marks are given for correct logic. Small syntax slips are penalised lightly; logic errors are not.
4. Do NOT use built-in functions that solve the whole task for you (for example sorted(), sum(), max() or min()) when a question says so.
5. Calculators are NOT allowed.
6. Number your answers exactly as the questions are numbered.

SECTION A

QUESTION 1: MULTIPLE CHOICE

Choose the correct answer. Write only the letter (A–D) next to the question number, e.g. 1.11 B.

1.1 Which of the following is a valid variable name in Python?

A 2total
B total_2
C total-2
D class

1.2 What is the value of the expression 17 // 5 ?

A 3.4
B 3
C 2
D 4

1.3 What is the value of the expression 17 % 5 ?

A 2
B 3
C 3.4
D 12

1.4 What is the data type of the value "42" ?

A int
B float
C str
D bool

1.5 What does len([3, [1, 2], 5]) return?

A 3
B 4
C 5
D 2

1.6 Which loop is the best choice when the number of repetitions is NOT known before the loop starts?

A A for loop over range()
B A while loop
C A nested for loop
D No loop can do this

1.7 What is printed by print(3 * "ab") ?

A ab3
B abab ab
C 3ab
D An error message

1.8 Which keyword sends a value back from a function to the code that called it?

A print
B return
C send
D output

1.9 Binary search can only be used when the list is …

A sorted.
B shorter than 100 items.
C made of whole numbers only.
D free of duplicate values.

1.10 In the worst case, linear search on a list of n items makes about … comparisons.

A 1
B $\log_2 n$
C n
D n^2

[10]

SECTION B

QUESTION 2: PROGRAMMING CONCEPTS

2.1 Explain the difference between a compiler and an interpreter. (2)

2.2 Name and briefly describe the THREE basic control structures used in structured programming. (3)

2.3 Distinguish between a syntax error, a runtime error and a logic error. Give an example of each. (3)

2.4 Explain the difference between a parameter and an argument of a function. (2)

2.5 Give TWO reasons why a program should be divided into functions. (2)

[12]

QUESTION 3: CODE TRACING

Study each piece of code and answer the questions. Show your working where it helps you.

3.1 Consider the code below.

```
x = 5
y = 3
x = x + y
y = x - y
x = x - y
print(x, y)
```

3.1.1 What is the output? (1)

3.1.2 In one sentence, what does this code do to x and y? (1)

3.2 Consider the code below.

```
total = 0
for i in range(1, 10, 2):
    if i % 3 == 0:
        continue
    total += i
print(total)
```

3.2.1 Draw a trace table with the columns i and total, showing every value that i takes. (3)

3.2.2 What is the output? (1)

3.3 Consider the function below.

```
def mystery(n):
    count = 0
    while n > 0:
        count += n % 10
        n //= 10
    return count

print(mystery(4072))
print(mystery(9))
```

3.3.1 What is the output? (2)

3.3.2 Give a suitable, meaningful name for the function mystery. (1)

3.4 What is the output of the code below? (4)

```
words = ["kasi", "braai", "taxi", "lekker"]
result = []
for w in words:
    if len(w) > 4:
        result.append(w.upper())
    else:
        result.append(w[::-1])
print(result)
```

3.5 What is the output of the code below? (3)

```
for row in range(1, 4):
    line = ""
    for col in range(row):
        line += str(row * col)
    print(line)
```

3.6 What is printed by the code below? Show the recursive calls. (2)

```
def f(n):
    if n <= 1:
        return 1
    return n * f(n - 2)

print(f(7))
```

[18]

QUESTION 4: DEBUGGING AND TESTING

The function below should return the average of the POSITIVE numbers in a list, and 0 if the list has no positive numbers. It contains THREE errors.

```
1  def average_positive(numbers)
2      total = 0
3      count = 0
4      for n in range(numbers):
5          if n > 0:
6              total = total + n
7              count = count + 1
8      return total / count
```

4.1 For EACH of the three errors, give the line number, the type of error (syntax, runtime or logic) and the correction. (6) (3 × 2)

4.2 Give TWO test cases (the input list and the expected result) for the corrected function. One of them must be an edge case. (2)

4.3 Explain why programmers test their code with edge cases. (2)

[10]

QUESTION 5: WRITING FUNCTIONS AND PROGRAMS

5.1 Write a function is_leap(year) that returns True if year is a leap year and False otherwise. A year is a leap year if it is divisible by 4, except years divisible by 100, which are leap years only if they are also divisible by 400. (5)

5.2 Write a function count_vowels(text) that returns the number of vowels (a, e, i, o, u, in upper or lower case) in the string text. (5)

5.3 A taxi rank records the number of passengers on each trip in a list, for example [15, 4, 12, 3, 15]. Write a function trip_summary(trips) that returns a tuple with: the total number of passengers, the largest number of passengers on one trip, and the number of trips with fewer than 5 passengers. Do NOT use sum(), max() or min(). For the example the function returns (49, 15, 2). (8)

5.4 A text file marks.txt has one line per student: a student number and a mark separated by a comma, for example 221045,67. Write a program that: (12)

- reads every line of the file;
- prints each student number followed by PASS (mark 50 or more) or FAIL;
- prints the class average, rounded to 1 decimal place, at the end;
- prints a suitable message, instead of crashing, if the file does not exist.

[30]

QUESTION 6: SEARCHING AND SORTING

6.1 Show the list [29, 10, 14, 37, 13] after EACH pass of a bubble sort that sorts it in ascending order. (4)

6.2 Write a function binary_search(items, target) that returns the index of target in the sorted list items, or −1 if target is not in the list. Use a loop, not recursion. (8)

6.3 The sorted list [3, 8, 15, 21, 30, 42, 56, 61, 77] is searched for 61 using binary search, with middle = (low + high) // 2. List, in order, the values that are compared with 61. (3)

6.4 Give the worst-case time complexity, in Big-O notation, of: (a) linear search, (b) binary search, (c) bubble sort. (3)

6.5 Explain why binary search is much faster than linear search for a sorted list of 1 000 000 items. (2)

[20]

TOTAL: 100

END OF QUESTION PAPER